RN

RouteNex

Dynamic logistics routing and fleet coordination hub

Capstone demo and viva package

Pitch · architecture · who can do what · demo plan with timings

Where the code lives · 56 viva questions with answers · honest limits

Prepared for: Sami Khan, B.Tech CSE, Jaipur National University

Rides · Parcels · Shop orders · Route optimizer · ML · AI assistant

October 2026 · read section 1 aloud before you start the demo

1. Say it first: the pitch

30 seconds

RouteNex is one platform for the three things a city needs moved: **people, parcels and shop orders**. Customers book from a phone app; drivers get jobs and earn; dispatchers plan efficient routes with a capacity-and-time-window optimizer; trained ML models predict ETAs, delay risk and demand; and an assistant lets anyone ask for things in plain language, with every change confirmed first.

2 minutes

Apps like Ola and Blinkit look simple on the surface, but underneath they need four things working together: **live dispatch** (who should serve this request), **fair and transparent pricing**, **route planning** that respects capacity and delivery windows, and **trust** (OTPs, refunds, role-based access). I built that stack end to end for Jaipur.

Customers can book Bike, Auto or Cab rides, send parcels with pickup and delivery OTPs, or order shop services, and pay by cash, wallet, UPI or card. Drivers see jobs and earnings. Dispatchers run a route optimizer that shows, before anything is saved, how it compares with the old nearest-neighbour method. Three ML models improve the ETAs shown at booking, flag stops likely to be late, and forecast demand. A built-in assistant, optionally powered by a free local or cloud model, can quote, track, book and cancel, but it can only *propose* changes: nothing happens until the user presses Confirm.

I will be clear about what is real and what is simulated, and I fixed several security problems I found in my own first version. Then I will show it.

What is real and what is simulated (say this before anyone asks)

Real	Simulated or demo-only
Booking, pricing, surge, coupons, OTP flows, cancel and refund rules	Drivers: in demo mode a driver autopilot moves trips along (can be switched off)
Database with a wallet ledger, atomic driver claiming, migrations	UPI and card payments (no money moves); wallet and refunds are a real ledger
Route optimizer on real orders, weights and windows; real road distances (OSRM)	ML training data is synthetic (no public per-trip dataset for Jaipur)
Role-based security, rate limits, escaped output, CSP	SMS OTP and Google sign-in run in demo mode only
Live weather feed; fleet, order and trip data inside the app	Fleet positions are demo coordinates around Jaipur

2. The project at a glance

Users and roles	Customer, Driver, Dispatcher, Fleet manager, Admin. Each gets its own screens; permissions are enforced on the server.
Services	Rides (Bike/Auto/Cab), parcels (bike courier, mini cargo, van), shop orders, wallet and coupons, driver console, route optimizer, analytics, assistant.
Backend	Python 3.12, FastAPI, pydantic validation, SQLite (WAL mode), about 66 API endpoints, about 2,900 lines in routers and services.
Frontend	Plain JavaScript single-page app, installable as a PWA, Leaflet maps, no build step.
Database	SQLite with foreign keys, indexes, additive migrations, a wallet ledger and an audit log.
Intelligence	Route optimizer (VRP heuristic), 3 scikit-learn models, retrieval-based FAQ, optional LLM (Ollama or Gemini free tier) with a built-in fallback.

Quality	80 automated tests: unit tests, API integration tests on a fresh database, security regression tests.
Deployment	Runs on a laptop; Docker file included; phone demo through a free Cloudflare tunnel.
Folders	`backend/` (API, services, tests), `frontend/` (app and PWA files), `database/` (schema, seed, tools), `ml/` (data generator, training, models), `docs/`.

Numbers you can quote (your Analytics page shows the exact values on your machine)

Topic	Result	Honest note
Automated tests	80	Browser screens and real LLMs are not covered by automated tests
ETA model	about 3.6 min average error vs about 7.8 min for the old formula	Synthetic data: proves the pipeline, not real-world accuracy
Delay-risk model	AUC about 0.91 (simple rule: about 0.87)	Synthetic data; probabilities are well calibrated on the test days
Demand model	about 26% lower error than 'same hour last week'	Synthetic data
Route optimizer	9 to 27% less distance than the old planner on synthetic benchmarks, lateness removed	Old planner ignored delivery windows, so cost gains are larger than distance gains

3. Architecture

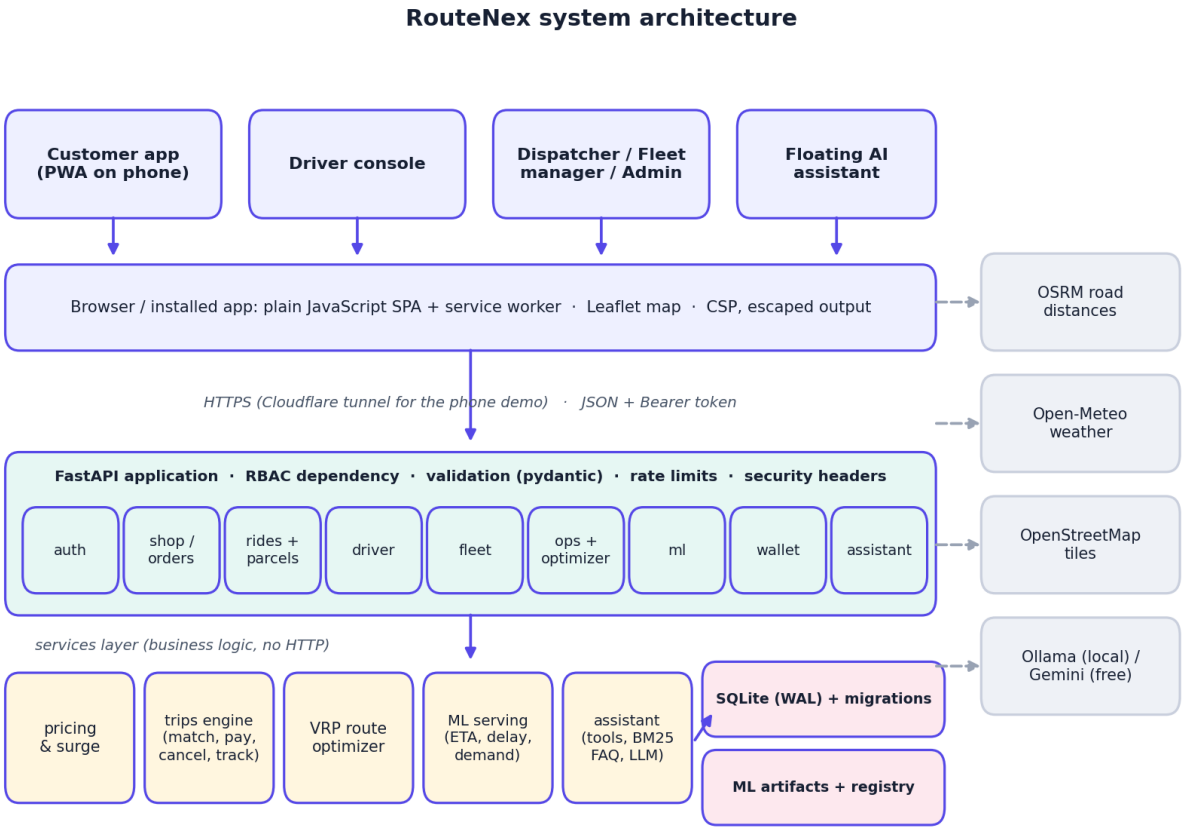


Figure 1. Clients talk to one FastAPI application; business logic lives in services; data sits in SQLite and ML artifacts; external services are optional.

How to explain it in 20 seconds: the browser app is thin; every rule (pricing, matching, payments, OTPs, permissions) is enforced on the server. Routers handle HTTP and validation, services hold the logic, the database holds the truth. External services are optional: without internet the app still works, with simpler distances and no live weather.

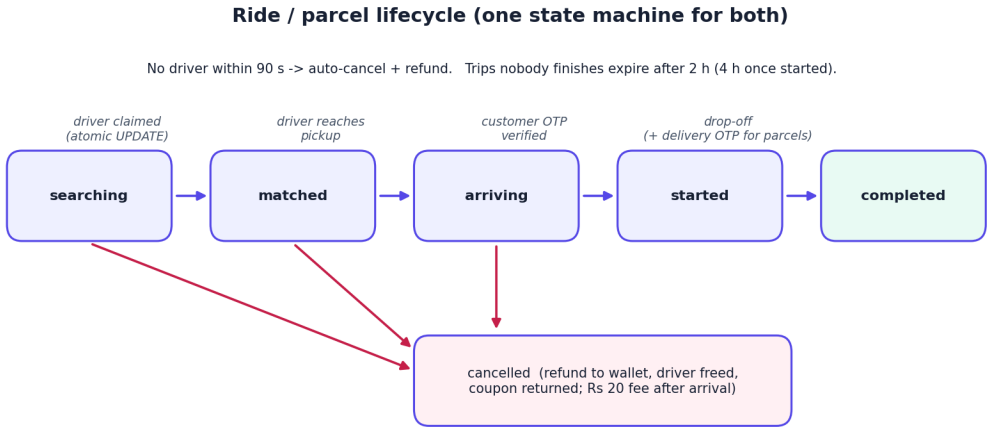


Figure 2. One state machine drives both rides and parcels. Parcels add a delivery OTP at the end.

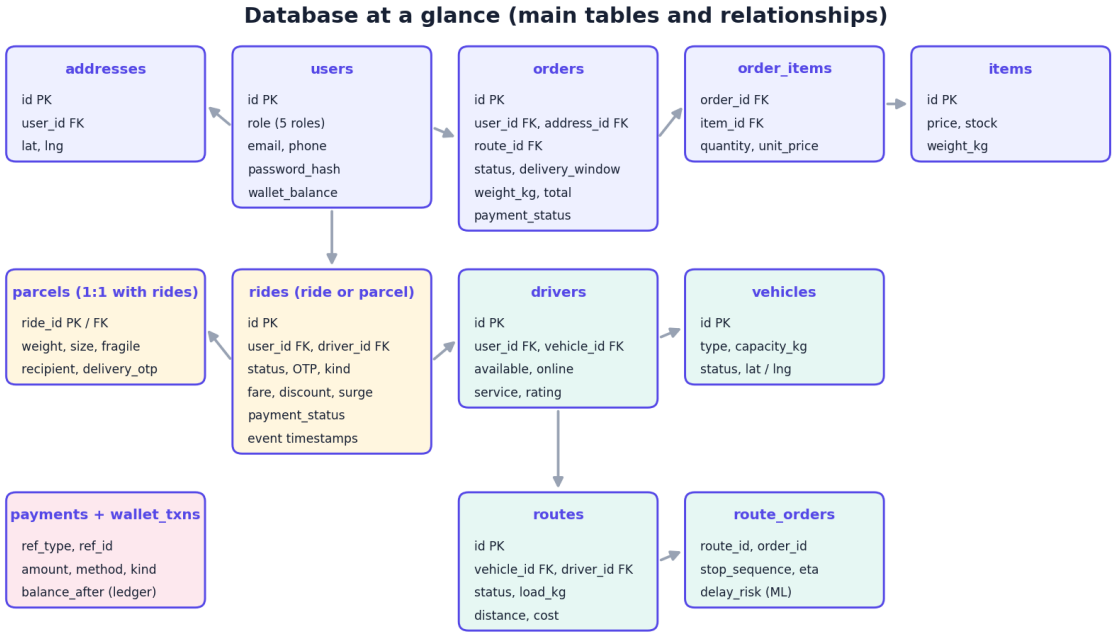


Figure 3. Main tables. Parcels extend rides 1:1; payments and the wallet ledger record every money movement.

Design decisions worth mentioning

- **Server decides, client only suggests.** The browser sends the route distance it measured; the server accepts it only if it is physically plausible and always computes the fare itself.
- **Atomic claims.** A driver is claimed with a conditional update (`WHERE available=1`) and the row count is checked, so two riders can never get the same driver. Stock is decremented the same way.
- **One transaction per business action.** Booking inserts the ride, charges the wallet, records the coupon and claims the driver together; if the wallet is short, nothing is saved.
- **Ledger, not a balance field alone.** Every wallet change writes a row with the balance after it.
- **Migrations that cannot lose data.** New columns are added only if missing, so an old database upgrades itself at startup.
- **Graceful degradation.** No scikit-learn: formulas. No internet: estimated distances. No LLM: built-in assistant engine.

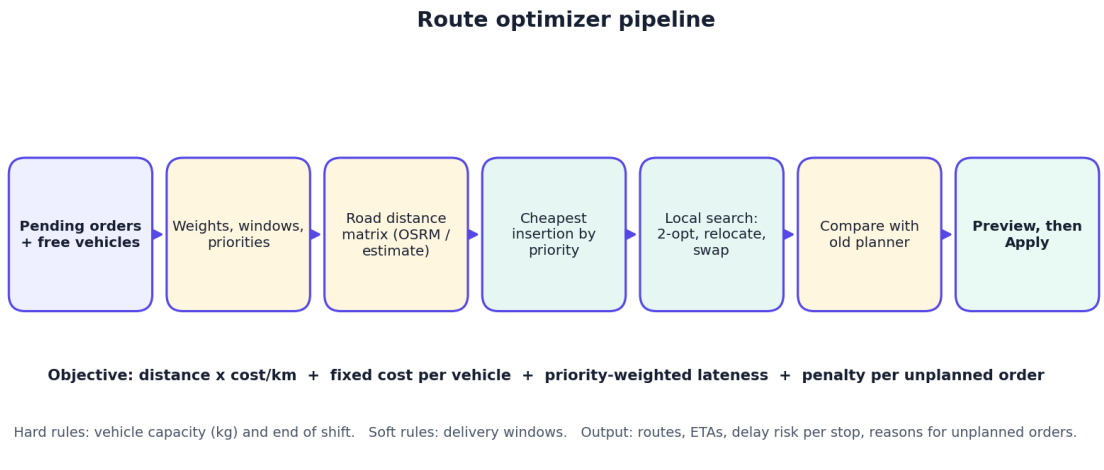


Figure 4. Route optimizer: plan, improve, compare, preview, then apply.

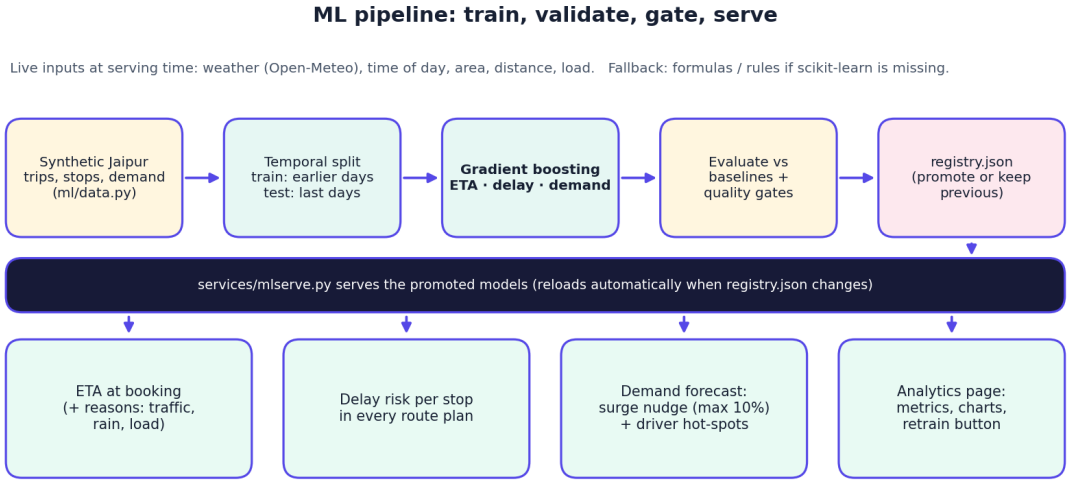


Figure 5. ML pipeline. Models are promoted only if they pass quality gates, then used in booking, routing, pricing hints and driver hints.

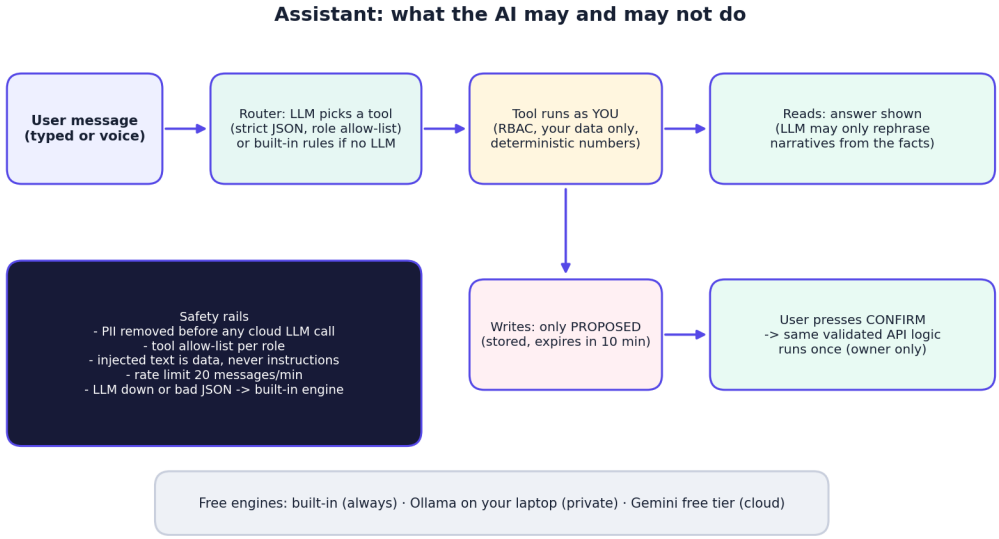


Figure 6. The assistant can read and propose; only the user can confirm a change.

4. Who can do what

Feature	Customer	Driver	Dispatcher	Fleet mgr	Admin
Book rides, parcels, shop orders; wallet; coupons	yes				
Live tracking, OTP, rating	yes				
Driver console, route stops, earnings, busy areas		yes			
Route optimizer, reroute, alerts			yes	yes	yes
Vehicles and drivers management			view	yes	yes
Analytics and ML dashboard, retrain models			view	view	yes
Users, activate/deactivate, audit trail					yes
AI assistant (tools differ by role)	yes	yes	yes	yes	yes

5. Demo plan

Accounts

Role	Login	Password
Admin (you)	samikhan75706@gmail.com	the password you set with create_admin (the old admin@routenex.in is switched off)
Customer	customer@routenex.in	Demo@123
Delivery driver (bike RJ14-RN-101)	driver@routenex.in	Demo@123
Dispatcher / Fleet manager	dispatcher@routenex.in / fleet@routenex.in	Demo@123
Other delivery drivers	driver2 ... driver12 @routenex.in	Demo@123. Vehicles in order: 101, 102, 201, 202, 203, 301, 302, 401, 501, 502, 601, 701
Ride drivers (for manual mode)	ridedriver1 ... ridedriver12 @routenex.in	Demo@123. 1-4 = Bike RJ14-RD-B01..B04, 5-8 = Auto A01..A04, 9-12 = Cab C01..C04

Never type or show your admin password on a projector. Sessions are per browser: use two browsers (for example Firefox for admin, Edge for driver) or private windows, plus your phone for the customer.

Preparation timeline

When	Do this
The day before	Run the tests (80 pass). Run `reset_demo.bat`, then `start.bat`, then `preflight.bat` until it says READY. Practise the 10-minute script twice. **Record a 3-minute backup video** of the full flow (Windows key + G). Copy the whole project folder to a pen drive. Charge laptop and phone.
30 minutes before	Laptop on power. Phone hotspot ready as backup internet. Start the app (`start.bat`) and wait for 'Application startup complete'. Run `preflight.bat`. Start `start-phone.bat` and open the new https link on the phone in Chrome (each run gives a new link, so an app installed from an old link will not work; use Chrome or reinstall).
5 minutes before	Log in each role in its own browser or window. Customer on the phone, admin and driver on the laptop. Browser zoom 110 to 125 percent. Close other tabs and notifications. Have the Analytics page and the code folder ready in the background.

10-minute demo

Min	Screen	Do	Say
0:00	Login page	Show the five roles; show the installed app on the phone	One app, role-based screens, installable on a phone.
0:45	Customer (phone): Ride	Pickup C-Scheme, drop Airport. Point at Bike/Auto/Cab, the 'AI ETA' reasons and the demand tag. Apply coupon WELCOME50, pay with wallet, Book	The server computes the price, the ML model gives the ETA and tells you why.
2:00	Tracking	Show the car moving, the OTP, driver card. Let the demo driver finish; rate the trip	Same state machine for rides and parcels; the driver is simulated for the demo.
3:00	Assistant (customer)	Type: send a parcel from Malviya Nagar to Amer Fort 2kg. Then: Ravi 9876543210. Press Confirm	It only proposes; nothing happens until I confirm.

Min	Screen	Do	Say
4:00	Driver (laptop)	Driver console: earnings, busy areas from the demand model, My Route	Drivers earn 80 percent of fares and see where demand is coming.
4:45	Admin: Optimizer	Preview plan. Read the Old method vs Optimizer table, the delay-risk badges and the map. Then Apply plan	Capacity, delivery windows and priorities. Preview never saves.
7:00	Reroute	Assistant (admin): a plate from the plan + broke down, Confirm. Show orders re-planned	Dynamic rerouting: released orders are re-planned immediately.
8:00	Analytics and ML	Forecast chart, ETA predicted-vs-actual, calibration, model registry. Say the data is synthetic	Temporal validation, baselines, quality gates. I do not claim real-world accuracy.
9:00	Trust and quality	Open http://localhost:8000/api/admin/users in a new tab: it refuses without a login (401). Show the passing tests and /docs	Roles enforced on the server; 80 tests; found and fixed real security bugs.
9:45	Close	Limits and next steps (section 10)	Real fleet data, payment gateway, PostgreSQL, WebSockets.

Extra beats if you have 20 minutes

- **Shop checkout:** add a service, coupon SHOP100, cash on delivery; then Optimizer picks it up; driver marks Picked up, Out for delivery, Delivered.
- **Wallet and refunds:** cancel a ride and show the refund line in the wallet ledger.
- **Admin:** deactivate a user and show that they can no longer log in.
- **Swagger at /docs:** show the typed API and one error message.
- **Phone app:** show install, then switch Wi-Fi off to show the offline banner.
- **Code walk (3 files):** `services/trips.py` (atomic claim), `services/vrp.py` (insertion + local search), `ml/train.py` (temporal split and gates).

Two-device live variant (impressive, needs practice)

In `backend\.env` set `DEMO_AUTOPILOT=false` and restart. Book a Cab on the phone: the booking tells you which driver was matched (for example Cab C02 = ridedriver10@routenex.in). Log in as that driver on the laptop: press **I have arrived**, enter the OTP the phone shows, then **Complete trip**. For parcels, the recipient's delivery OTP is also shown on the phone. Switch back to `true` afterwards.

If something goes wrong

Symptom	Likely cause	Fix
Map is grey	No internet or tiles blocked	Switch laptop to phone hotspot; run preflight
'No driver online' or nothing matches	Demo fleet not created	Restart the app (demo fleet is created at startup) or run reset_demo.bat
'You already have an active ride'	An earlier test trip is still open	Open it from the banner and Cancel (old trips also expire by themselves)
Optimizer says no orders	Everything already planned	Place 3 orders as a customer, or run reset_demo.bat
Analytics says models not trained	First start still training (about 10 s)	Wait, or admin presses Retrain models
Assistant says 'built-in engine'	No LLM installed	That is fine; it still works. Optional: install Ollama

Symptom	Likely cause	Fix
Phone cannot open the link	Old link, or class Wi-Fi blocks it	Use the newest link from start-phone.bat; use hotspot
Login fails with 429	Too many wrong attempts	Wait 5 minutes or restart the app
Everything is broken	Anything	Restart start.bat; if still broken play the backup video and explain the design

6. Where is the code?

Topic	Files
Auth, passwords, tokens, roles	`backend/security.py`, `backend/deps.py`, `backend/routers/auth.py`
Ride and parcel lifecycle, payments, coupons	`backend/services/trips.py`, `backend/routers/rides.py`, `backend/routers/wallet.py`
Pricing and surge	`backend/services/pricing.py` and `market()` in `trips.py`
Shop orders and stock	`backend/routers/shop.py`
Route optimizer	`backend/services/vrp.py`, `routers/ops.py` (`plan_and_apply`), `services/roads.py`, `services/windows.py`
ML models	`ml/train.py`, `ml/data.py`, `ml/features.py`, `backend/services/mlserve.py`, `routers/ml.py`
Assistant	`backend/services/assistant.py`, `llm.py`, `kb.py`, `places.py`, `routers/assistant.py`
Database	`database/schema.sql`, `backend/db.py` (migrations), `database/seed.py`
Frontend	`frontend/app.js` (base), `phase2.js` (customer, driver), `phase3.js` (optimizer, route), `phase4.js` (analytics), `phase5.js` (assistant), `sw.js`, `manifest.webmanifest`
Tests	`backend/tests/` (9 files, 80 tests)
Tools	`start.bat`, `start-phone.bat`, `preflight.bat`, `reset_demo.bat`

Read these five files before the viva (you will be asked about them)

File	What to be able to explain
`security.py`	Password hashing, token signing and expiry, rate limiter
`services/trips.py`	Atomic driver claim, charge and refund, the state machine, the demo autopilot
`services/vrp.py`	The objective, insertion, local search moves, the baseline comparison
`ml/train.py`	Temporal split, baselines, metrics, quality gates, registry
`services/assistant.py`	Tool routing, role allow-list, pending actions and confirmation, fallback

If you used AI tools to help build this, check your institution's rules on disclosing that, and be ready to explain every module yourself. Reading the five files above, running the demo yourself, and answering the questions below out loud is what makes the project yours.

7. Viva question bank

Answer in your own words; the answers below are the facts and the shape. Short, honest answers beat long ones. When you do not know, say what you would check.

A. Start here: the questions almost everyone gets

Q. Walk me through booking a ride from tap to arrival.

The app sends pickup, drop, vehicle class, payment and coupon to `POST /api/rides`. The server validates the input, **recomputes the distance and fare itself** (the browser's number is only a hint), applies live demand pricing and the ML ETA, then in **one transaction** checks the customer has no active ride, applies the coupon, charges the wallet if chosen, inserts the ride and claims the nearest eligible driver with a conditional update. The screen then polls the tracking endpoint every 2 seconds. The driver arrives, the customer gives a 4-digit OTP, the driver starts and completes; cash is settled or the prepaid amount is kept, the driver is freed, and the customer can rate.

Q. What is real and what is simulated in your project?

Real: the booking logic, pricing, the database and wallet ledger, OTP rules, the optimizer, role-based security, live weather and road distances. Simulated: the drivers (an autopilot for the demo), UPI and card payments, and the ML training data, because there is no public per-trip dataset for Jaipur.

Q. What was the hardest part?

Two things: making booking safe under concurrency (atomic driver claim and stock updates inside transactions) and the optimizer (capacity, windows and priorities together). I also spent real effort finding and fixing security problems in my first version.

Q. How is this different from Ola or Blinkit?

They have real fleets, payments licences and years of data. RouteNex is not a competitor; it demonstrates the full stack behind such apps in one project: dispatch, pricing, routing, ML and an assistant, with the safety rules that make them trustworthy.

Q. What would you do with two more weeks?

Collect real trip data to retrain the models, add WebSocket tracking instead of polling, move to PostgreSQL with Redis, integrate a real payment gateway in test mode, benchmark my optimizer against OR-Tools, and add CI plus browser tests.

B. Architecture and engineering

Q. Why FastAPI?

Type hints give automatic request validation (pydantic) and an automatic OpenAPI page at `/docs`; it is fast, simple to test with its test client, and keeps routers small.

Q. Why a plain JavaScript front end with no framework?

No build step, so one folder runs everywhere, and it installs as a PWA easily. The trade-off is larger hand-written files; for a bigger team I would move to React or Vue with components.

Q. What are routers and services?

Routers translate HTTP to function calls and enforce roles and validation. Services hold the business rules and know nothing about HTTP, so the same logic is reused by the API, the assistant and the tests.

Q. How does the PWA work?

A manifest makes it installable; a service worker caches the app files (network first, so updates arrive), caches map tiles, and never caches API calls because they are private and live. Offline it shows a banner and saved screens.

Q. What breaks first if 10,000 users arrive?

SQLite allows one writer at a time, the rate limiter and caches live in one process, and tracking uses polling. I would move to PostgreSQL, Redis (rate limits, locations), WebSockets, and a queue for notifications, behind several workers.

Q. How did you test it?

80 automated tests: unit tests for the optimizer, ML and assistant parsing; API tests that run against a fresh database for every test; and security regression tests (for example that a customer cannot cancel someone else's order). Browser screens and real LLMs were checked by hand.

C. Security

Q. How are passwords and sessions protected?

Passwords use PBKDF2-SHA256 with 210,000 iterations and a random salt per user. Sessions are signed, expiring tokens (HMAC-SHA256). Roles are checked in a dependency on every protected endpoint, and ownership is checked per record.

Q. What security bugs did you find in your first version?

A demo Google sign-in that trusted a client-supplied email (anyone could log in as admin), fares computed from browser-supplied distance, missing ownership checks on rides and orders, unescaped data in the page (XSS), a `.env` file that was never loaded, and no rate limiting. All are fixed and covered by tests.

Q. How do you prevent XSS and CSRF?

All server data is escaped before display, inline event handlers were removed, and a Content-Security-Policy limits scripts to our own files and the map library. The token is sent in a header, so classic CSRF does not apply; I store it in localStorage, which I would replace with an httpOnly cookie plus a CSRF token in production.

Q. Why a custom token instead of a JWT library?

To stay dependency-free and to show how signing and expiry work. In production I would use a vetted library such as PyJWT or Authlib and add key rotation.

Q. How is brute force limited?

Failed logins are limited per IP and email (8 in 5 minutes), OTP requests and wrong OTP attempts are limited, trip OTPs lock after 5 wrong tries, and assistant messages are limited to 20 per minute. The limiter is in memory, so with several workers it would move to Redis.

Q. Are OTPs stored safely?

Login OTPs are stored as keyed hashes (HMAC) with a 5-minute expiry, old codes are invalidated, and the fixed demo OTP works only in demo mode. Trip OTPs are 4-digit codes visible only to the customer who owns the trip.

Q. How are payments modelled?

Cash and cash-on-delivery settle on completion. The wallet is a real ledger: debit is a conditional update (`WHERE balance >= amount`) plus a transaction row with the balance after. UPI and card are simulated. Refunds always go to the wallet.

D. Database (your DBMS strengths)

Q. Which normal form is the schema in, and where did you denormalise?

Mostly 3NF: separate tables for users, addresses, items, orders and order lines, with composite keys such as `route_orders`. I deliberately store snapshots where history matters: `unit_price` in order lines, `fare` and `discount` on rides, so later price changes do not rewrite the past. Parcels are a 1:1 extension table of rides.

Q. Where do you use transactions and what do they guarantee?

Booking, cancelling, order placement and optimizer apply each run in one transaction (atomic): either every step happens or none. A test checks that a failed wallet charge leaves no ride and frees the driver. Isolation comes from SQLite serialising writers; durability from write-ahead logging.

Q. How do you stop two riders getting the same driver?

A compare-and-set: `UPDATE drivers SET available=0 WHERE id=? AND available=1` and checking that exactly one row changed. In PostgreSQL I would use `SELECT ... FOR UPDATE SKIP LOCKED` to pick and lock a driver in one step.

Q. Why SQLite, and how would you move to PostgreSQL?

Zero setup and good enough for a demo. For production: PostgreSQL for concurrent writers, proper types, row locking and replication. The SQL is mostly standard; I would replace SQLite's JSON helper in a few queries and use a migration tool such as Alembic.

Q. What indexes did you add and why?

On the columns used in filters and joins: orders by user and status, rides by user and driver and status, routes by vehicle and driver, route stops by route, wallet rows by user, OTP rows by phone. They keep the dashboards and the matching queries fast as data grows.

Q. How do migrations work?

A list of (table, column, definition). At startup each column is added only if `PRAGMA table_info` says it is missing, so any older database upgrades itself without losing data, and a fresh database goes through the same path.

Q. Is the audit log really immutable?

No. It is append-only by convention (the application never updates or deletes it) but a database admin could change it. A truly immutable log needs write-once storage or hash chaining.

E. Distributed database ideas applied to this project

Q. How would you shard or replicate RouteNex?

Partition by city (then by zone) because a request only touches one city; keep users and wallets in a strongly consistent primary; store live driver locations in an in-memory geo store; use read replicas for analytics and the ML dashboard.

Q. Which parts need strong consistency and which can be eventual (CAP)?

Wallet balance, booking and driver claiming need consistency (CP): a wrong balance or a double booking is unacceptable. Live map positions, demand forecasts and analytics can be eventually consistent (AP): a few seconds of staleness is fine.

Q. How would a booking that spans services (trip service and payment service) stay correct?

Not with two-phase commit, which blocks. I would use a saga: reserve the driver, charge the wallet with an idempotency key, confirm the trip, and run compensating actions (refund, release the driver) if a step fails, using an outbox table to publish events reliably.

F. Route optimization

Q. What problem does the optimizer solve?

A capacitated vehicle routing problem with soft time windows and priorities: assign waiting orders to available vehicles and order the stops so that distance, lateness and unplanned orders are minimised, without exceeding vehicle capacity or the end of shift. It is NP-hard, so exact methods do not scale.

Q. What algorithm did you use?

Orders are sorted by priority and inserted at the cheapest feasible position (cheapest insertion); then local search repeats three moves while they reduce cost: 2-opt inside a route, relocating one order anywhere, and swapping two orders between routes. A time limit (1.5 s by default) bounds the work.

Q. What exactly is the objective?

Distance times the vehicle's cost per km, plus a fixed cost per vehicle used, plus a priority-weighted lateness penalty per minute, plus a large penalty for every order left unplanned. Capacity and end of shift are hard rules.

Q. How do you know it is better, and what is the 'old method'?

I compare against the previous planner (nearest neighbour, capacity by weight, ignores windows) on the same distance matrix. On synthetic benchmarks distance dropped 9 to 27 percent and lateness disappeared, and a test checks the optimizer is never worse than the baseline. The cost gain is larger than the distance gain because the old planner ignored windows.

Q. Why not Google OR-Tools?

I wanted something I could explain line by line, with no heavy dependency. OR-Tools would likely find better solutions on large instances; benchmarking against it is my first next step.

Q. How are time windows and overdue orders handled?

Windows are read from the order text. They are soft: arriving late costs money but is allowed. A window that has already passed becomes 'serve as soon as possible' for 90 minutes; a window opening more than 2 hours away is left for a later plan.

Q. Where do distances come from?

The public OSRM table service gives real road distances when online; offline the app uses straight-line distance times 1.3, and the page tells you which source was used.

Q. What happens when a vehicle breaks down?

Its unfinished orders are released, the route is marked reroute-required, and the optimizer re-plans them over the remaining vehicles in the same request.

Q. What are the limits of the optimizer?

No pickup-and-delivery pairs, vehicles start from their current position, travel times use a simple speed model, and there is no multi-day planning. Solutions are good, not guaranteed optimal.

G. Machine learning

Q. What do the models predict and with what algorithm?

ETA in minutes (regression), probability a delivery window is missed (classification), and requests per area per hour (Poisson regression). All use scikit-learn gradient boosting, which handles non-linear interactions in tabular data, needs no feature scaling and is fast.

Q. Where does the training data come from? Is it real?

It is synthetic: a generator in `ml/data.py` simulates Jaipur trips with traffic by hour and area, rain, vehicle type and load, plus noise and unobservable factors. I say so on the Analytics page. There is a hook for real data. The metrics show the pipeline works and beats baselines; they do not claim real-world accuracy.

Q. Why trust the numbers then?

For real-world accuracy you should not. What they demonstrate is correct method: temporal validation, baselines, no leakage, calibration checks, quality gates and explanations. With real data the same code would give real numbers.

Q. How did you avoid data leakage?

The split is temporal (train on earlier days, test on the last days), and every feature is known at prediction time: slack is computed from the plan, traffic level is a static profile of area and hour, weather is current or forecast.

Q. Explain the metrics.

MAE is the average minutes of error; R-squared is the share of variance explained. AUC is the chance the model ranks a late stop above an on-time one; Brier score measures probability quality; the calibration chart checks that '30 percent risk' really means about 30 percent of those stops were late. For demand, MAE versus 'same hour last week'.

Q. What are quality gates and the registry?

After training, each model must beat its baseline by a threshold (for example ETA at least 5 percent better and R-squared at least 0.5). Only then it is promoted in `registry.json`; otherwise the previous version stays live. The last five versions are kept.

Q. How do you explain a prediction?

For ETA and delay risk the app re-predicts with one factor neutralised (peak traffic, rain, load, tight window) and reports the biggest differences as plain text such as 'Heavy traffic +4 min'. The Analytics page also shows permutation importance.

Q. How would you monitor and retrain in production?

Log predicted versus actual ETAs and delays, watch error and calibration drift by week, retrain on a schedule with the same gates, and roll back by pointing the registry at the previous file. The Retrain button demonstrates the loop.

Q. Where is ML actually used in the product?

ETA with reasons at booking, a delay-risk percentage on every stop of a route plan (saved with the route), a demand forecast used for driver hot-spot hints and a price nudge of at most 10 percent, and live weather as an input.

Q. Is dynamic pricing fair?

It is capped (at most 1.8 times), shown to the customer as a visible demand tag before booking, and based on supply and demand and time of day, not on personal data.

H. The assistant (LLM)

Q. How do you stop the assistant making things up?

The model never writes prices, balances or statuses. It only chooses a tool (strict JSON validated against a per-role allow-list) and, for narrative answers, rephrases facts the tool returned. Policy answers come from a small retrieval step (BM25) over written passages. If the model fails or returns invalid JSON, a built-in rule engine answers.

Q. What about prompt injection?

A message cannot make the assistant do more than the user could do: tools run with the user's own role and data, write actions are only proposed and need the user to press Confirm (each proposal belongs to one user, expires in 10 minutes and runs once), and tool results are given to the model as data. A test shows a customer cannot reach an operations tool even when the model is told to.

Q. What about privacy with cloud models?

The default is the built-in engine or a local Ollama model, so nothing leaves the laptop. If a Gemini key is set, phone numbers and emails are removed before sending; the free tier may be used by Google to improve products, so it should not be used with real customer data.

Q. Why BM25 and not embeddings (RAG)?

The knowledge base is 16 short passages, so keyword ranking works, is explainable and needs no downloads. With hundreds of documents I would add embeddings and a vector index.

Q. How do you evaluate the assistant?

Automated tests cover routing, confirmation, role limits, redaction and fallback using a scripted stand-in for the model. I have not benchmarked routing accuracy of a specific real model; small local models can mis-route, which is why numbers are deterministic and changes need confirmation.

Q. Why a free local model?

Zero cost and full privacy. The trade-off is quality: a 3-billion-parameter model is less reliable than a large cloud model, which the design compensates for with validation and confirmation.

I. Trap questions

Q. Is the payment real? Is the ML real?

Wallet and refunds are a real ledger; UPI and card are simulated. The ML code and pipeline are real; the data is synthetic. I would rather tell you that than have you find out.

Q. Did you build all of this yourself?

Answer truthfully according to your institution's rules. If you used AI assistance, say that you used it as a tool, that you reviewed and ran everything, and then demonstrate understanding by explaining the files in section 6 and making a small change if asked.

Q. Can you change something right now?

Be ready for a tiny live change, for example the fare constants in `services/pricing.py`, the cancellation fee `CANCEL_FEE` in `services/trips.py`, or a new FAQ passage in `services/kb.py`. Practise one of these.

8. Limits and future work (say them yourself)

Limit today	What production would add
Drivers, GPS and positions are simulated	Driver app with real GPS, WebSocket live tracking
UPI and card are mocked; SMS OTP and Google sign-in are demo-only	Payment gateway (test mode first), SMS provider, verified Google ID tokens
SQLite, one worker; rate limits and caches in memory	PostgreSQL, Redis, several workers, a job queue
Token in localStorage; custom token code	httpOnly cookies, CSRF tokens, a vetted JWT library, key rotation
ML trained on synthetic data	Real trip and GPS logs, drift monitoring, scheduled retraining
Optimizer: heuristic, no pickup-delivery pairs, simple travel-time model	Compare with OR-Tools, use the ETA model inside the optimizer, multi-day planning
Assistant routing quality depends on the chosen free model	Evaluation set of real messages, larger or hosted model, Hindi coverage in the built-in engine
No automated browser tests	End-to-end tests (Playwright), CI pipeline

9. Command cheat sheet

Task	Command (run in the project folder)
Start the app	double-click `start.bat`, open http://localhost:8000
Phone link	double-click `start-phone.bat` (new https link every time)
Check everything before the demo	with the app running: double-click `preflight.bat`
Clean demo state	stop the app, then double-click `reset_demo.bat` (recreates your admin login)
Run all tests	`"%LOCALAPPDATA%\RouteNexVenv\Scripts\python.exe" -m pytest backend\tests -q`
Create or fix the admin login	`"%LOCALAPPDATA%\RouteNexVenv\Scripts\python.exe" database\create_admin.py --name "Sami Khan" --email samikhan75706@gmail.com --retire-demo-admin`
Retrain the ML models	admin: Analytics & ML, Retrain models (or `python ml\train.py`)
Manual driver mode (two-device demo)	set `DEMO_AUTOPILOT=false` in `backend\.env`, restart
Optional local AI	`winget install Ollama.Ollama`, then `ollama pull qwen2.5:3b`, restart the app